E-COMMERCE BACKEND SYSTEM

Capstone Project



ENSP456 – New Age Programming Languages Lab

By

ANJALI (2301420021)
JIGYASA SINGH (2301420032)
BHUVEETA SAROHI (2301420011)
ARSHIYA TAHIM(2301420031)
DEV PAWAR(2301420028)
LAKSHAY PAWAR(2301420007)

Submitted to- Dr. Hemraj Kumawat(SOET)
Semester- 7th
Course - Bachelor of Technology in Computer Science and Engineering(Data Science)
Section - A
Submitted on – 10 November 2026

1. Project Overview

This project is a working online-store backend that follows one customer from account registration, through browsing and the shopping cart, to checkout and order history. A browser storefront talks to a set of small Go services through a single gateway. PostgreSQL holds the business data. When an order is placed, the Go order service passes a message through Redis to a Python worker, which produces the invoice, the notification and the recommendation data after the customer has already received a confirmation.

The Phase I report set out the design. Since then the system has been built, run and tested. This report describes what exists, how it was checked, the problems met along the way, and what remains for Phase III.

2. Objectives

The objectives carried over from Phase I are:

1. Provide a browser storefront and a documented REST API.
2. Split account, product, inventory and order responsibilities into separate Go services.
3. Store users, products, stock, carts and orders permanently in PostgreSQL.
4. Keep checkout correct when many customers order at once, so that no more units are sold than are in stock.
5. Move slower post-order work (invoices, notifications, recommendations) out of the checkout request and into a background worker.

3. Technologies Used

Technology	**Use in the project**
Go 1.24	Gateway and the user, product, inventory and order services, using the standard net/http package
Python 3.12	Asynchronous worker (asyncio) with invoice, notification and recommendation handlers
PostgreSQL 16	Accounts, catalogue, stock, carts, orders, notifications and co-purchase counts
Redis 7	Publish/subscribe channel (order_events) that carries the OrderCreated event
ReportLab	PDF invoice generation in the worker
HTML, CSS, JavaScript	Storefront, written without a build step and served by the gateway
Docker Compose	Builds and starts the database, Redis, Go services, worker and gateway together

4. Implementation and System Flow

The gateway is the only part of the system reachable from the browser. It serves the storefront files, checks the login token on protected requests and forwards each request to the service that owns it. Each service checks the token again for itself, so a request that skips the gateway is still refused. The Go services share one PostgreSQL database, which keeps stock and orders inside a single transaction.

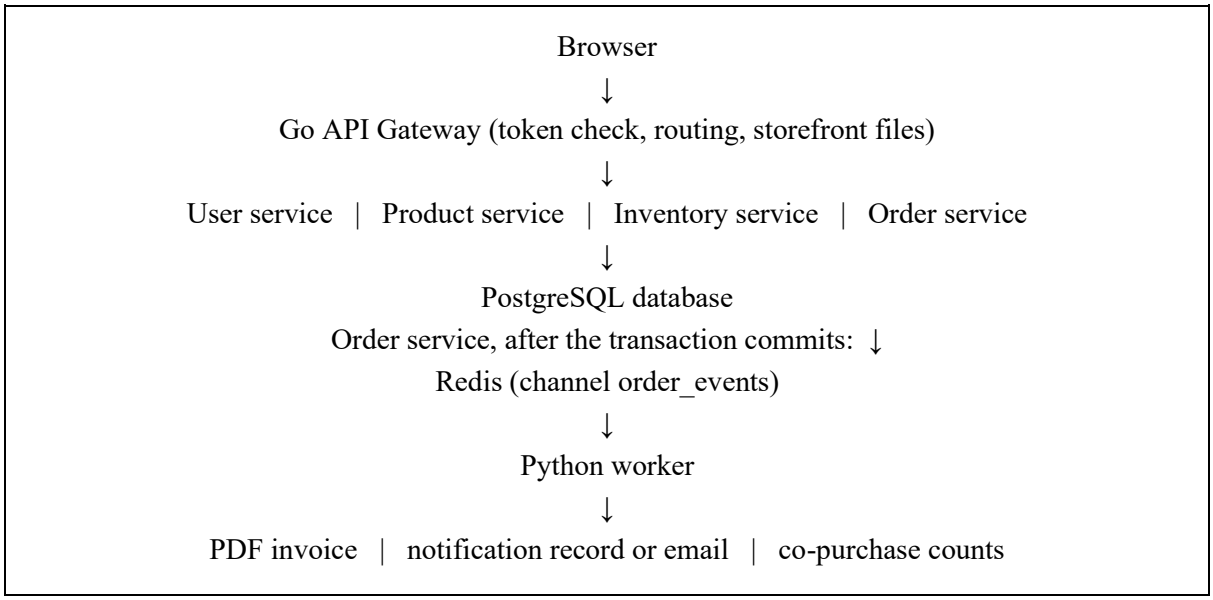


Figure 1. Request path and background processing

4.1 Checkout

Checkout runs as one database transaction. The order service first locks the customer's cart rows, which stops a double-clicked button from creating two orders. It then reduces stock with a conditional update that only succeeds while enough units remain, writes the order and its items, and empties the cart. If any step fails, the whole transaction is rolled back. Stock rows are always taken in product order, so two customers buying overlapping items cannot block each other in a deadlock. Payment is simulated and no real payment method is charged.

4.2 Background processing

Once the transaction has committed, the order service publishes an OrderCreated event to Redis and returns the order to the customer. The Python worker receives the event and runs its three handlers together. The invoice handler builds the PDF in a separate thread and renames the file into place only when it is complete, so a half-written invoice is never served. The notification handler stores a message for the user and sends an email when SMTP settings are present. The recommendation handler adds one to a counter for every pair of products that appear in the same order. A failure in one handler is logged and does not stop the others.

5. Progress Against the Work Plan

Milestone	Planned work	Status	Remaining
2	Repository, Docker setup, database, service skeletons	Complete	None
3	User, product and inventory services with REST endpoints	Complete	Go unit tests
4	Order service, checkout, Redis publishing	Complete	None
5	Python worker: invoice, notification, recommendation	Complete	Reliable event delivery
6	Integration and concurrency testing	Mostly complete	Docker run on a clean machine, load results in final form
7	Final documentation and viva preparation	In progress	Screenshots, final report

6. Testing and Results

Two scripts in the tests folder exercise the system through the gateway, as a customer would. The results below come from a run of the current code on Ubuntu 24.04 with Go 1.24, PostgreSQL 16, Redis 7 and Python 3.12, with every service started by the project's local run script.

Test	Result
End-to-end smoke test: registration, wrong password, duplicate email, catalogue, cart limits, checkout, order history, empty cart, unknown product and invoice	18 of 18 checks passed
50 shoppers each order 1 unit of a product that has 20 units	20 orders accepted, 30 refused with a stock error, final stock exactly 0
50 shoppers place 4 orders each with plenty of stock	200 of 200 orders accepted in 1.24 seconds, about 161 orders per second; final stock matched the expected value
Background output after 221 test orders	221 PDF invoices and 221 notification records
A second customer requests another customer's invoice	Refused (404)
Login token with a changed signature	Refused (401)

The throughput figure was measured with the database, Redis, all services and the test client sharing one machine, so it shows how the system behaves under modest load and should not be read as a production benchmark.

To run the project: docker compose up --build. The storefront is then available at http://localhost:8080.

7. Problems Met and How They Were Handled

The first design question was how to stop two customers buying the last unit at the same moment. Checking stock in the application and then updating it leaves a gap in which both requests pass the check. The final design moves the check into the update itself, which the database runs one request at a time for a given row. The 50-shopper test confirms the result.

A second problem was the invoice. Generating the PDF is slow compared with a database write, and an early read of the file could return half a document. Rendering now runs in a thread, away from the event loop that receives orders, and the file is written under a temporary name and renamed when finished. Until it exists, the invoice endpoint answers that the invoice is still being prepared.

The services also start in an unpredictable order under Docker, and a service that starts before PostgreSQL would fail. Each Go service now retries its database connection for up to thirty seconds, and the Compose file waits for the database and Redis to report healthy before starting anything that depends on them.

8. Current Limitations and Planned Work

Payments are simulated, and email is sent only when SMTP details are configured; otherwise the notification is stored without an email. These match the scope set in Phase I.

A review of the code and a test run found two weaknesses that matter more. First, any signed-in user can change product prices and stock levels, because the system has no administrator role. Second, Redis publish/subscribe does not keep messages, so an order placed while the worker is stopped never receives its invoice or notification, although the order itself is saved correctly. Smaller gaps are the lack of Go unit tests, the lack of a login rate limit, and an empty recommendation list for customers with no history.

The plan for Phase III is, in order:

1. Add an administrator role and restrict product and inventory changes to it.
2. Make event delivery reliable, using a transactional outbox or Redis Streams, and make the worker handlers safe to run twice for the same order.
3. Write Go unit tests for token handling, password hashing, input checks and stock reservation, and run them with the other tests.

4. Add a login rate limit and refuse to start when the token secret is missing or left at its default.
5. Show popular products to customers who have no purchase history.
6. Recod the final test results and complete the final report and viva preparation.

9. Application Interface

The figures below show the running application. Each was captured from the system started with the command in Section 6.

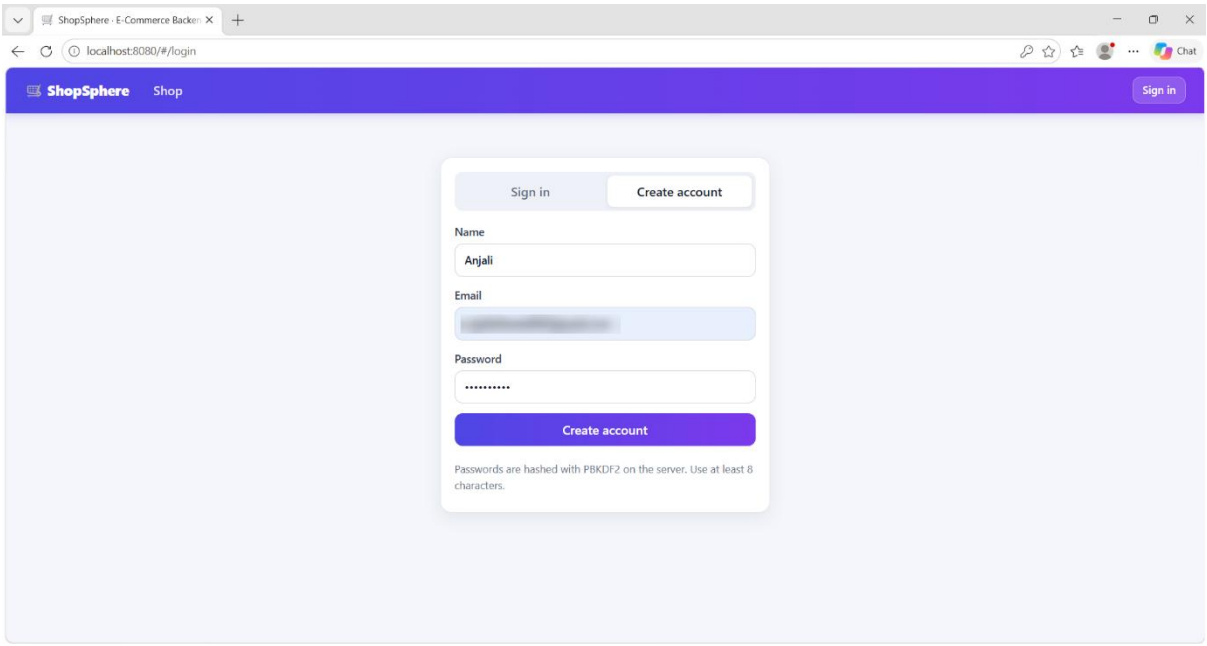


Figure 1. Creating an account

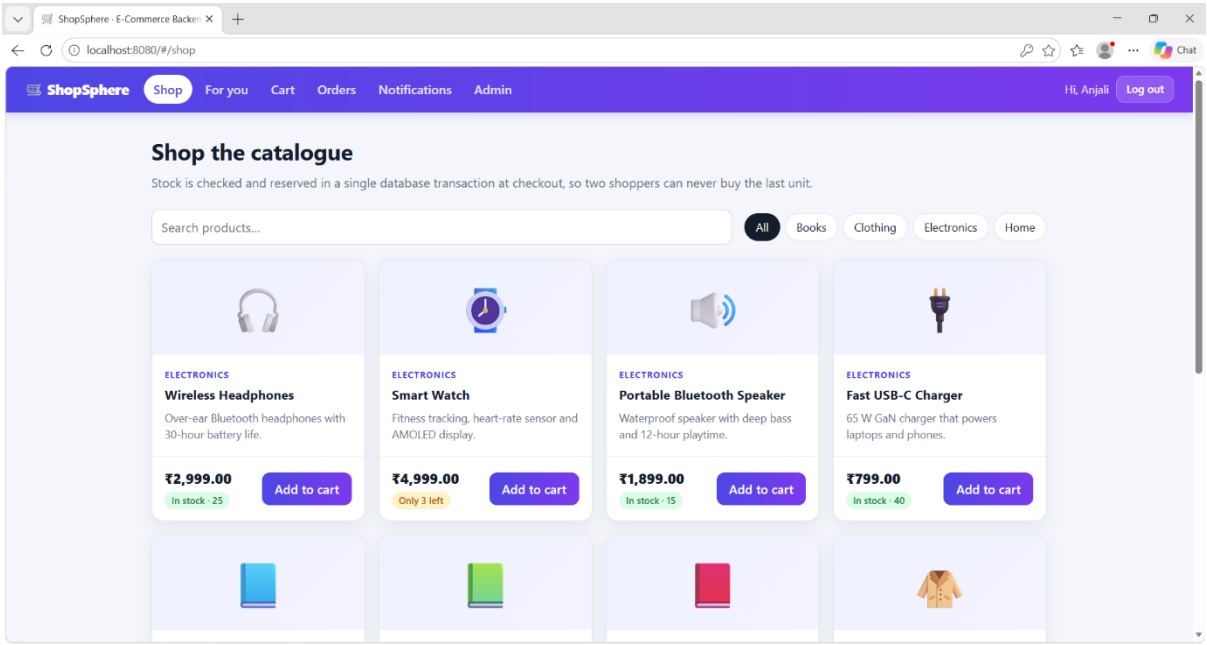


Figure 2. Storefront and product catalogue

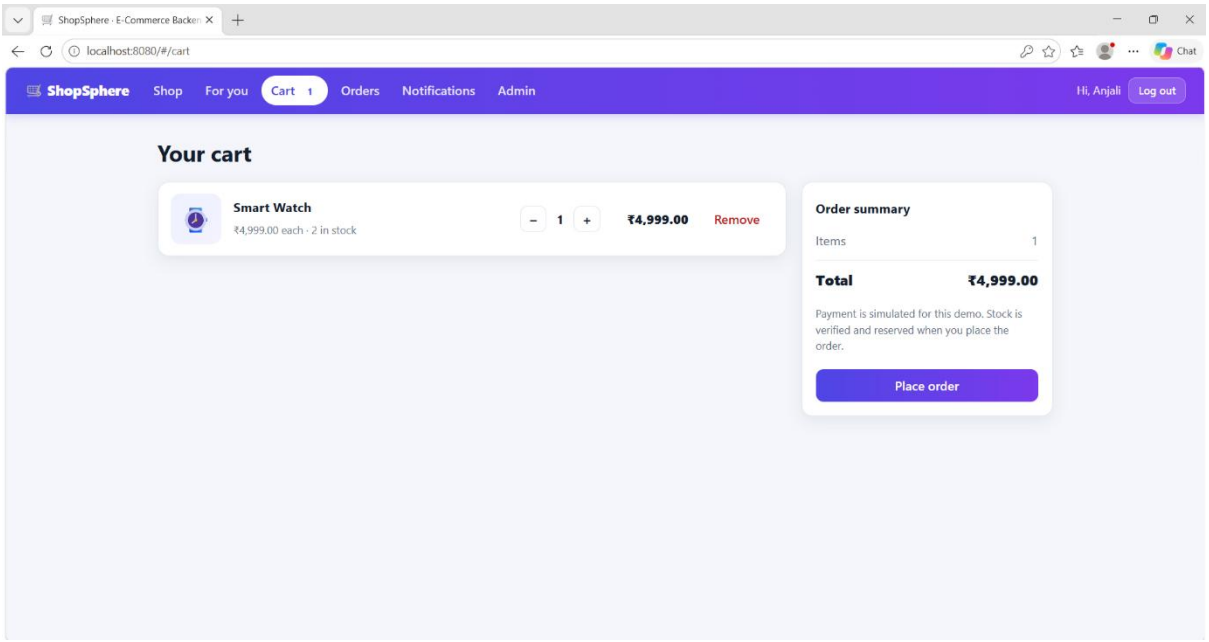


Figure 3. Shopping cart and checkout

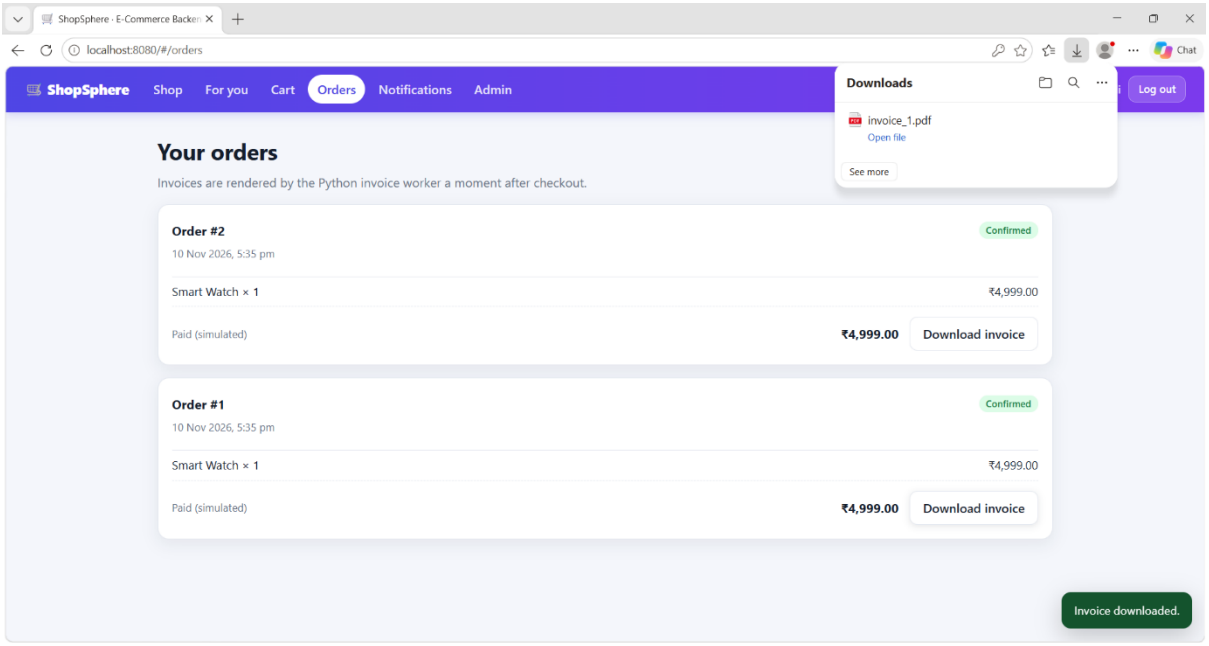


Figure 4. Order history and invoice

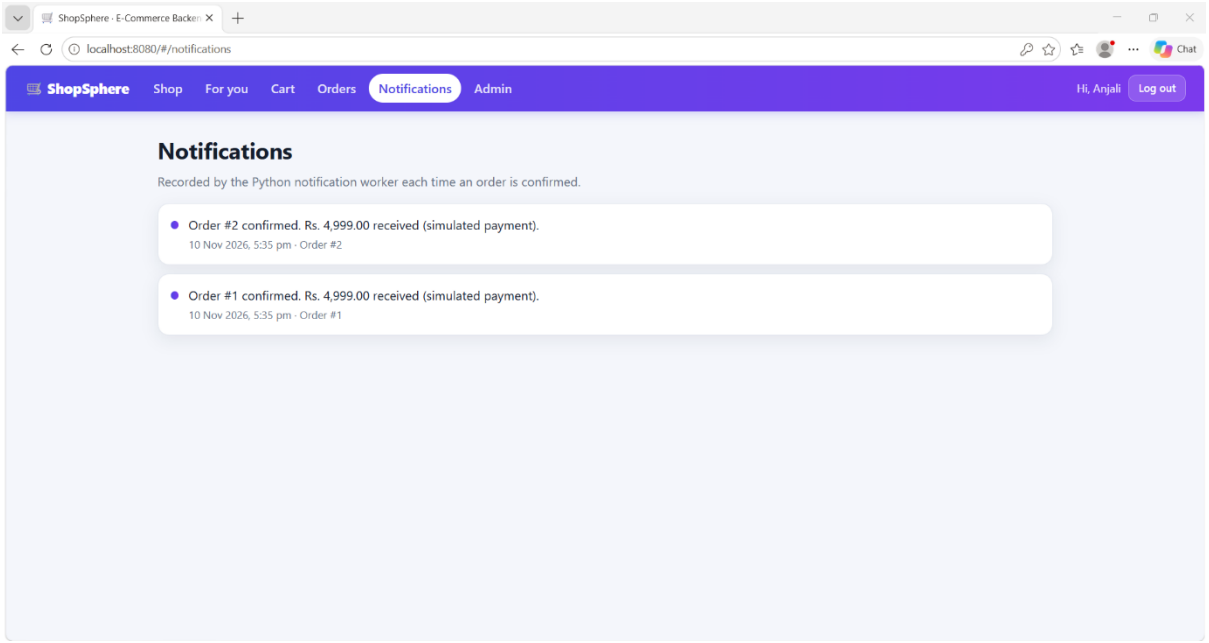


Figure 5. Notifications